TECHNICAL INTERVIEW PREP · 1 DE 10

Technical Interview Questions - Part 1 - Javascript

Giuliana Olmos · 17 de octubre de 2024 · https://dev.to/giulianaolmos/technical-interview-questions-part-1-javascript-5gcj

Introduction

Hey everyone!

How's it going?

I've been thinking about starting a series based on the theoretical questions I've come across in technical interviews. I'll be covering *Javascript, NodeJS, Typescript, and SQL*.

The goal of this series is to give quick responses and explain key concepts. Also, you'll notice "Part 1" in the title—I'm not sure yet how many parts this series will have! 😄

This first chapter will dive into ✨Javascript✨.

So, let's go...

Questions

1. What are the differences between regular and arrow functions in JavaScript?
2. What are the differences between `then()` and `async/await`?
3. What timing events do you know in JS, and how do they work?

Question 1: What are the differences between regular and arrow functions in JavaScript?

Answer:

> - They are syntactically different (Arrow functions were introduced in ECMAScript 6).
> - ~~Arrow functions can be anonymous (they don't need a name).~~ *//Clarification in comments*
> - The key difference lies in how they handle the `this` context.

In this post, I'll focus on the most commonly asked aspect: the difference in how `this` is handled.

Let's use this object as an example.

```
const cat = {
 name: 'Rubi',


//This is a regular function
 regularGreet() {
   console.log(`Hello, my name is ${this.name}. I mean... meow`);
 },


//This is an arrow function
 arrowGreet: () => {
   console.log(`Hello, my name is ${this.name}. I mean... meow`);
 },
};
```

If you run those functions, the result will be the next.

```
cat.regularGreet(); // return "Hello, my name is Rubi. I mean... meow"

cat.arrowGreet(); // return "Hello, my name is undefined. I mean... meow"
```

In the arrow function, `${this.name}` is `undefined.`

> DISCLAIMER: This example works if you're using Node or Deno. However, it will throw an error in Bun because the default context is `undefined`.

But...

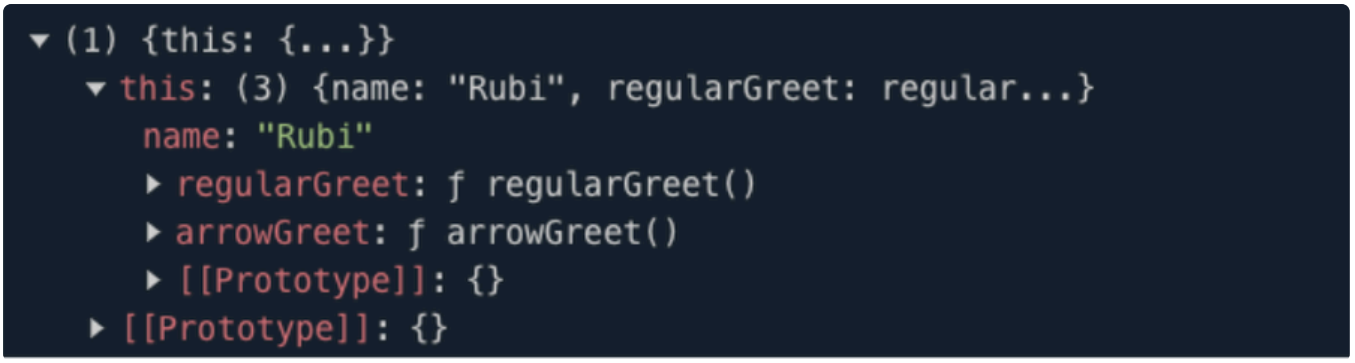
Because of how the `this` keyword works:

In regular functions, `this` refers to the object the function is a part of. However, arrow functions don’t have their own `this`; instead, they inherit it from the surrounding context.

For example, if I log `this` inside a regular function, it will output the cat object.

```
 regularGreet() {
   console.log({ this: this });
   console.log(`Hello, my name is ${this.name}. I mean... meow`);
 },
```

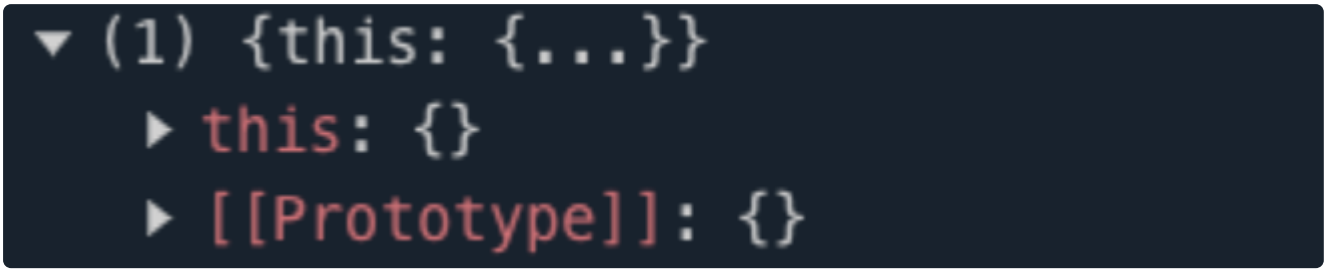
Log:



But, when I log `this` in an arrow function:

```
arrowGreet: () => {
   console.log({ this: this });
   console.log(`Hello, my name is ${this.name}. I mean... meow`);
 },
```

Log:



`This` is empty.

> It's essential to know that when we don't have a broader scope, `this` refers to the Window object.

How could you print the name in `arrowGreet`?

Well, you could define `this`, inside `arrowGreet()`

```
arrowGreet: () => {
  console.log({ this: this });
  this.name = 'Rubi'


  console.log(`Hello, my name is ${this.name}. I mean... meow`);
},
```

Now, you will be logging:

```
Hello, my name is Rubi. I mean... meow
```

However, this is not a good practice; if you have to define this inside the function, use regular functions instead.

But if you want to inherit the context for the arrow function. You could do the next.

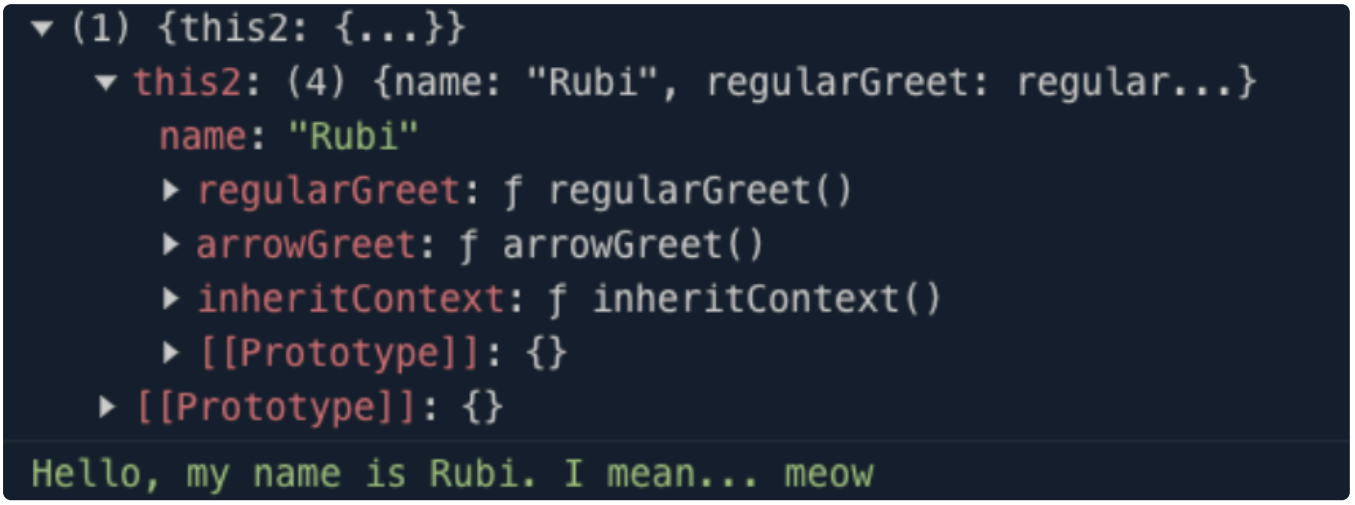
```
    inheritContext() {
      const inceptionArrow = () => {
        console.log({ this2: this });
        console.log(`Hello, my name is ${this.name}. I mean... meow`);
      };
      inceptionArrow();
    },
```

`inheritContext()` => It's a regular funcion. So the context of `"this"` is the same as `regularGreet(),` the object cat.

`inceptionArrow()` => It's inside of `inheritContext()`. As we said before, the arrow function inherits its context. So, which context is it inheriting from?

Yes, cat.

The result of printing `this2`



We can apply the same approach with another example, this time using `setTimeout()`. In this case, we have a regular function that contains an arrow function, which inherits its context from `secondCombinationFunctions()`.

```
    secondCombinationFunctions() {
      setTimeout(() => {
        console.log({ this3: this });
        console.log(`Hello, my name is ${this.name}. I mean... meow`);
      }, 1000);
    },
```

Question 2: What are the differences between `then()` and `async/await`?

Answer:

> There are no differences between them other than their syntaxes.

If we have an existent function called `asyncFunction()` and we run it using `async/await` and `then()`

```
await asyncFunction();
console.log('Run this after running asyncFunction');


asyncFunction.then(() => {
 console.log('Run this after running asyncFunction');
});
```

The results and the order of its run will be the same.

> The difference could be that sometimes we need to run several functions concurrently, and using `then()` could create a callback hell.

```
function1().then(()=>{
  function2().then(()=> {
    function3().then(() => {
      function4...
    })
  })
})
```

Question 2.5: How do you avoid a callback Hell?

Answer:

> Using `async/await.` Avoiding nested `.then()` calls.

The execution is almost the same, but it’s more readable in this way.

`async/await:`

```
await function1()
await function2()
await function3()
```

```
        await function4()
        ...
```

`.then():`

```
        function1()
          .then(function2)
          .then(function3)
          .then(function4)
```

Question 3: What timing events do you know in JS, and how do they work?

Answer:

> I know about two different timing events in JS.
>
> `setTimeout()`
>
> `setInterval()`

Both methods from the `“window”` object allow you to run your code in intervals.

The definition of both from `w3school` is:

setTimeout(function, milliseconds) Executes a function, after waiting a specified number of milliseconds.

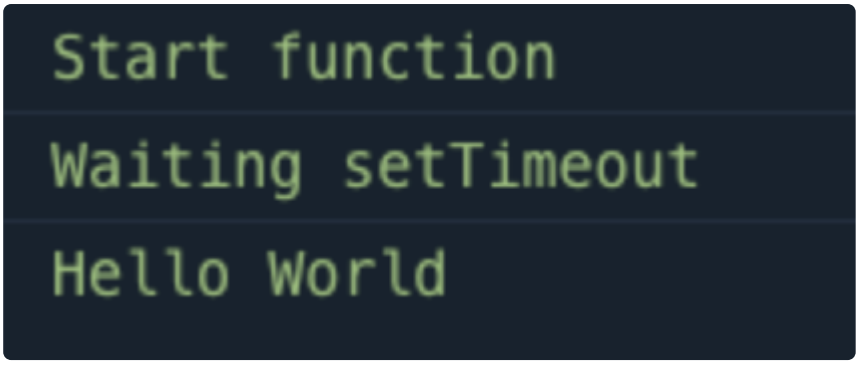
setInterval(function, milliseconds) It is the same as `setTimeout()` but repeats the function's execution continuously.

How it works setTimeout?

```
        function testSetTimeout() {
         console.log('Start function');
         setTimeout(() => {
           console.log('Hello World');
         }, 3000);
         console.log('Waiting set Time out');
        }


        testSetTimeout();
```

Logs:



The order of how it runs is

1. Run `console.log('Start function')`;
2. Run `setTimeout()`; The timing starts to count now before running the function inside setTimeout.
3. Run `console.log('Waiting set Time out')`;
4. The timer hits 3000 and runs the function that shows the `console.log`.

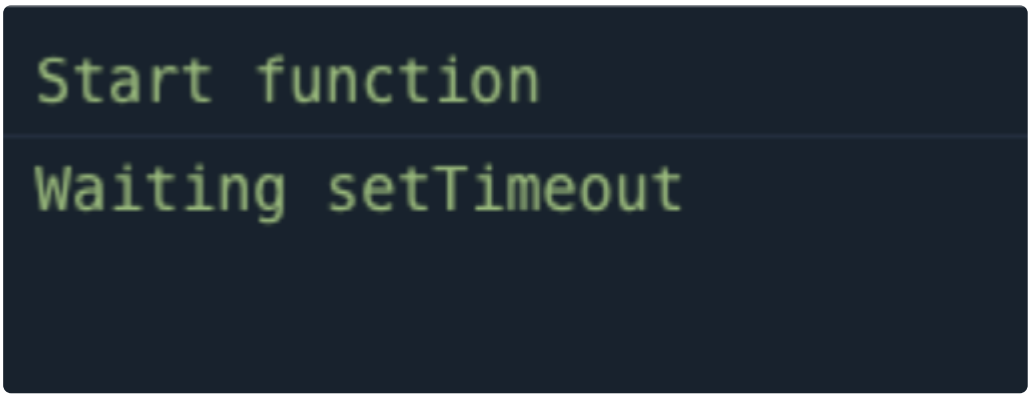
How do I stop the `setTimeout`?

When the function set with the `setTimeout` has yet to run, you can stop it with `clearTimeout()`.

```
function testSetTimeout() {
 console.log('Start function');
 const settedTimeout = setTimeout(() => {
   console.log('Hello World');
 }, 3000);
 console.log('Waiting setTimeout');
 clearTimeout(settedTimeout);
}


testSetTimeout();
```

Log:



How it works setInterval?

```
function testSetInterval() {
 console.log('Start function');
 setInterval(() => {
   console.log('Run interval');
 }, 2000);
 console.log('Waiting for interval starting');
}


testSetInterval()
```

1. Run `console.log('Start function');`
2. Run `setInterval();` As `setTimeout()` in that line is when the timer starts to count.
3. Run `console.log('Waiting for interval starting');`
4. The timer hits 2000ms and runs the function that shows the `console.log`. *Then, every 2000ms, the function runs again.*

```
Waiting for interval starting
Run interval
Run interval
Run interval
Run interval
Run interval
Run interval
Run interval
```

How could we stop it?

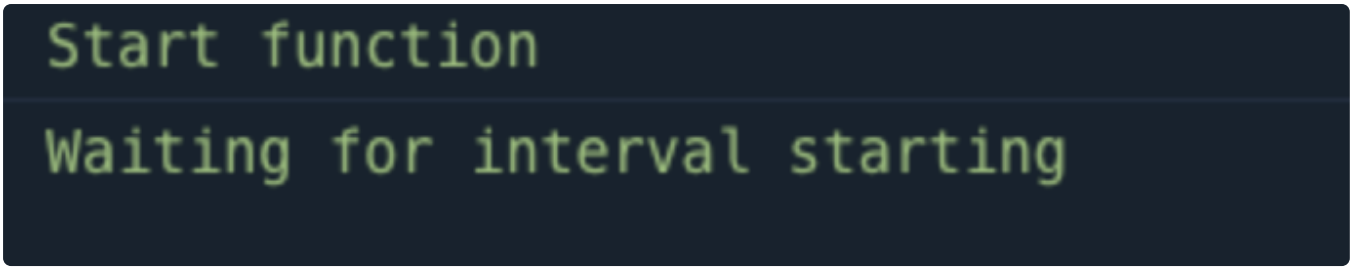
Using `clearInterval()`

```
function testSetInterval() {
 console.log('Start function');
 const settedInterval = setInterval(() => {
   console.log('Run interval');
 }, 2000);
 console.log('Waiting for interval starting');

 clearInterval(settedInterval);
}

testSetInterval()
```

The result is the next:



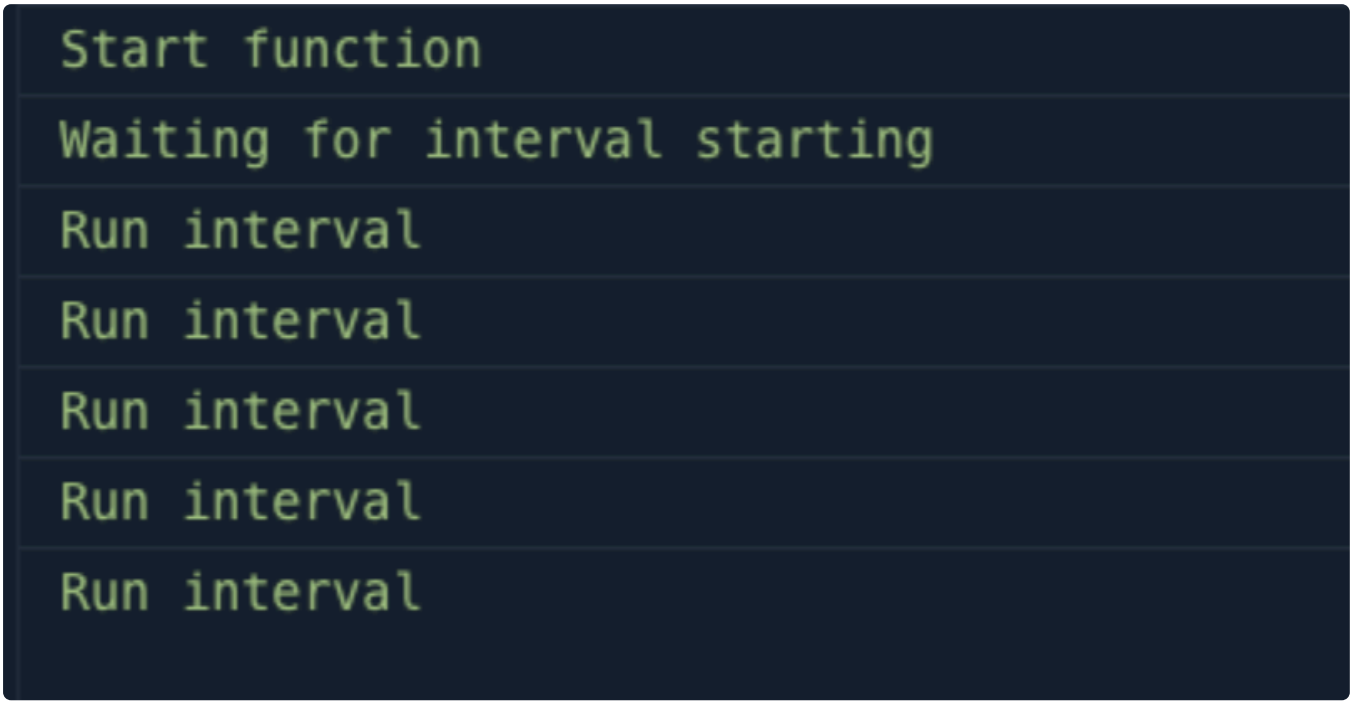
Or... we could combine the timers to step the interval after a specific time.

```
function testSetInterval() {
 console.log('Start function');
 const settedInterval = setInterval(() => {
   console.log('Run interval');
 }, 2000);
 console.log('Waiting for interval starting');


 setInterval(() => {
   clearInterval(settedInterval);
 }, 10000)
}


testSetInterval()
```

Then, the interval will run for 10k milliseconds for each 2k ms.



The end...

That’s all for now! I’m working on more posts covering common interview questions (I’ve got a list of others to tackle). If there’s a specific question you’d like me to explain, feel free to drop it in the comments, and I’ll make sure to include it.

✨I hope this is helpful to everyone!✨

Thank you so much, and see you next time 🫶

Last but not least

I want to say thanks to Samuel Rouse for taking the time to write comments, correcting and adding clarifications that helped me improve this post.